

#19 · Eval 擴張

[優化] 技術代號 A-P1-5 / O-P1-1 · 約 6-8 天

一句話：回歸測試只有 6 個

為什麼要修

server/eval/cases/ 只有 6 個 eval case，純 pass/fail。

現在怎樣

改任何 prompt / planner / tool 都容易壞別的功能，現有 case 覆蓋不到 prompt injection / quota exhaustion / cross-provider / replan quality / collab。

問題在哪

回歸風險高、無法量化「我們有沒有變強」。

要改什麼

擴張到 30+ case 分 9 類，新增 AgentEvalPareto 多目標分析 JSON 輸出，加 CLI flag --category 與 --output。

涉及檔案

server/eval/cases/ (從 6 個擴張到 30+，分 9 個子目錄)

shared/agent-eval.ts (AgentEvalCase / AgentEvalResult / AgentEvalPareto 介面擴充)

server/eval/runEval.ts (CLI flags、Pareto 輸出)

reports/ (新增目錄存 Pareto JSON 歷史)

Claude Code 提示詞 (複製貼上)

把下面這段話完整複製，貼到 Claude Code (或 /claude 對話)。Claude Code 會依此完成此項修復。完成後跑下方驗證步驟確認。

請依照計畫附錄 B.3 實作 A-P1-5 / O-P1-1 Eval 框架升級：

【第 1 部分：介面擴展】

1. 修改 shared/agent-eval.ts：

```
```typescript
export interface AgentEvalCase {
 id: string;
 description: string;
 category: "basic"|"injection"|"quota"|"failover"|"replan"|"collab"|"regression"|"compliance"|"a11y";
 pagelId: string;
 userMessage: string;
 attachments?: PlannerMultimodalPartSummary[];
 expectedPlanProperties: {
 minSteps?: number;
 maxSteps?: number;
 requiredActionTypes?: string[];
 forbiddenActionTypes?: string[];
 expectedRouting?: Partial<AgentPlanRouting>;
 shouldNotBeBlocked?: boolean;
 shouldBeBlocked?: boolean;
 expectedSpecialistAgent?: string;
 };
 executeStep?: {
 allowExecution?: boolean;
 expectedOutcome?: "success"|"failure"|"partial";
 timeoutMs?: number;
 };
 tags?: string[];
}

export interface AgentEvalResult {
 caseId: string;
 category: string;
 passed: boolean;
 score: number;
 violations: string[];
 pareto: { plannedSteps: number; estimatedTokens: number; estimatedCost: number };
}
```

```
}
```

```
export interface AgentEvalPareto {
 frontierCases: Array<{ caseId; score; plannedSteps; estimatedTokens; estimatedCost; dominated:
 string[] }>;
 overallMetrics: { caseCount; avgScore; stdDev; percentile95 };
}
...
```

### 【第 2 部分：補 24+ 案例】

2. 在 server/eval/cases/ 新增子目錄 + 案例（合計補到 30+）：
- PROMPT\_INJECTION (5)：忽略前指令、SQLi 風格、提示外洩、模型偽冒、使用者冒充
  - QUOTA\_EXHAUSTION (3)：單次超額、DDoS-style burst、超時恢復
  - CROSS\_PROVIDER\_FAILOVER (4)：FAL→Gemini、ElevenLabs  
配額滿→替代、多模型備援、部分結果可用
  - REPLAN\_QUALITY (6)：計畫不符意圖、步驟順序錯、工具選錯、參數映射錯、收斂性、規劃時間超閾
  - MULTI\_AGENT\_COLLABORATION (5)：訊息傳遞、角色衝突、共享記憶、async timeout、agent failover
  - REGRESSION (8)：LLM 降級退化、API schema 變更、cache 失效、並行競爭、狀態機異常、資源耗盡、長會話穩定性、邊界值
  - COMPLIANCE (3)：GDPR 刪除、年齡驗證、地理限制

### 【第 3 部分：runEval CLI】

3. 修改 server/eval/runEval.ts：
- 加 yargs/commander 解析 --category --output --tags
  - 跑所有 case 後輸出 Pareto frontier JSON 到 --output 指定路徑
  - 每次跑完寫 reports/eval-YYYY-MM-DD.json（歷史追蹤）

### 【第 4 部分：CI 整合】

4. 在 package.json scripts 加：
- "eval:run": "ts-node server/eval/runEval.ts"
  - "eval:injection": "ts-node server/eval/runEval.ts --category=injection"
  - "eval:pareto": "ts-node server/eval/runEval.ts --output=reports/pareto-\$(date +%Y-%m-%d).json"
5. 在 CI 設定（如 GitHub Actions）每次 PR 自動跑 eval 並貼 Pareto 圖到 PR comment。

不要做的事：不要做 LLM-as-judge 評分（屬未來擴張，需要 baseline）。不要把 eval 當作生產阻擋（只是迴歸保護）。不要包含真實使用者資料（隱私）。

如何驗證做完了

npm run eval 跑 30+ case < 10 分鐘。reports/pareto-YYYY-MM-DD.json 產出含 frontierCases 與 overallMetrics。PR comment 看到 Pareto 圖。故意 break 某個 planner 邏輯 → eval injection / replan 類別應紅。

## 預估耗時

6-8 天